# UART Core Specification

*by*

**Hossam Shaher Anis**

Cairo, 2026

# Contents

UART Core Specification ..... 1
Contents ..... 2
List of Figures ..... 3
List of Tables ..... 4
1 Overview ..... 5
1.1 Block diagram ..... 5
2 Parameters and Ports ..... 6
2.1 Parameters ..... 6
2.2 Ports ..... 6
3 Functionality ..... 7
3.1 UART Frame ..... 7
3.2 Baude Rate Generator ..... 7
3.3 UART Receiver (Rx) ..... 8
3.4 UART Transmitter (Tx) ..... 9
4 References ..... 11

# List of Figures

**Figure 1** Block diagram of the UART core ..... 5
**Figure 2** UART frame (8 data bits, no parity, and 1 stop bit) ..... 7
**Figure 3** A waveform of `tick` ($fclk = 1.8432\ MHz$ and $b = 9600\ bps$) ..... 8
**Figure 4** Receiving two UART frames ..... 9
**Figure 5** Transmitting two UART frames ..... 9
**Figure 6** Expanded view at the first `tx_start` pulse ..... 10
**Figure 7** Expanded view at the second `tx_start` pulse ..... 10

# List of Tables

**Table 1** Parameters of the UART core ..... 6
**Table 2** Ports of the UART core ..... 6

# 1 Overview

Serial communication uses a single data line (***serial line***) to exchange information between two systems.

The transmitting system converts the parallel data to a serial stream, and the receiving system reassembles the serial data back to its original parallel format.

A **UART (Universal Asynchronous Receiver and Transmitter) core** is the most commonly used scheme.

## 1.1 Block diagram

Figure 1 shows the block diagram of the UART core.

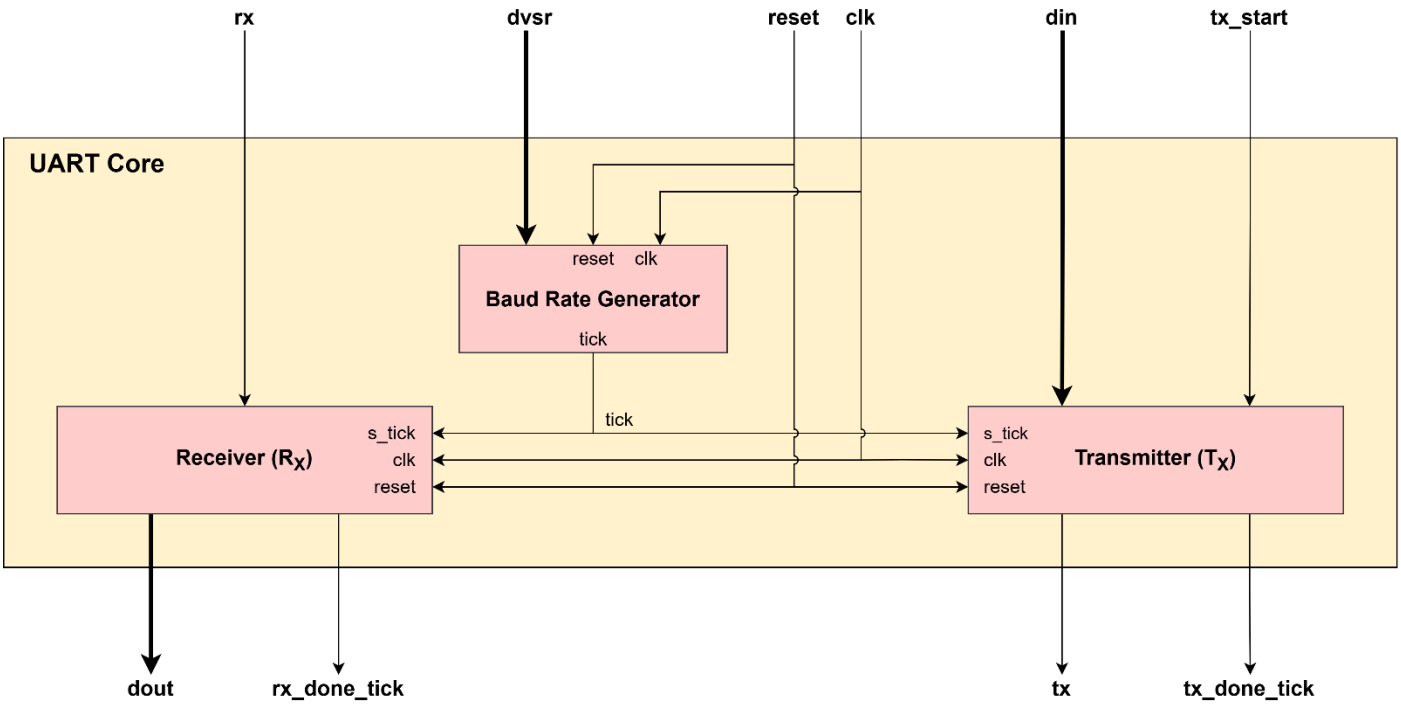


**Figure 1** Block diagram of the UART core

# 2 Parameters and Ports

## 2.1 Parameters

To allow future modification, two parameters are used in the design. See Table 1.

**Table 1** Parameters of the UART core

| Parameter Name | Default Value | Description |
|---|---|---|
| DBIT | 8 | Number of data bits. |
| SB_TICK | 16 | Number of ticks needed for the stop bits,<br>which is 16, 24, and 32 for 1, 1.5, and 2 stop bits, respectively. |

## 2.2 Ports

The UART core has three interfaces:

(1) System interface.
(2) Receiver ($R_X$) interface.
(3) Transmitter ($T_X$) interface.

Ports of the UART core are listed in Table 2.

**Table 2** Ports of the UART core

| Port Name | Interface | Width | Direction | Description |
|---|---|---|---|---|
| clk | System, $R_X$, $T_X$ | 1 | Input | System clock. |
| reset | System, $R_X$, $T_X$ | 1 | Input | Asynchronous active high reset. |
| dvsr | System, $R_X$, $T_X$ | 11 | Input | Run-time configurable divisor.<br>This signal controls the baud rate. |
| rx | $R_X$ | 1 | Input | Serial data input from the communication link. |
| dout | $R_X$ | 8 | Output | 8-bit received data. |
| rx_done_tick | $R_X$ | 1 | Output | A status signal.<br>It is asserted for one clock cycle after the receiving process is completed. |
| din | $T_X$ | 8 | Input | 8-bit data to be transmitted. |
| tx_start | $T_X$ | 1 | Input | A control signal.<br>It is asserted for one clock cycle to latch din and initiate transmission. |
| tx | $T_X$ | 1 | Output | Serial data output to the communication link. |
| tx_done_tick | $T_X$ | 1 | Output | A status signal.<br>It is asserted for one clock cycle after the transmission process is completed. |

# 3 Functionality

The UART core consists of three components (see Figure 1):

- The baud rate generator.
- The receiver (Rx).
- The transmitter (Tx).

## 3.1 UART Frame

The serial line is `1'b1` when it is ***idle***.

The transmission starts with a ***start bit***, which is `1'b0`, followed by ***data bits*** and an optional ***parity bit***, and ends with ***stop bits***, which are `1'b1`.

- The number of data bits can be 6, 7, or 8.
- The optional parity bit is used for error detection.
    - For odd parity, it is set to `1'b0` when the data bits have an odd number of 1's.
    - For even parity, it is set to `1'b0` when the data bits have an even number of 1's.
- The number of stop bits can be 1, 1.5, or 2.

The transmission with eight data bits, no parity, and one stop bit is shown in Figure 2.

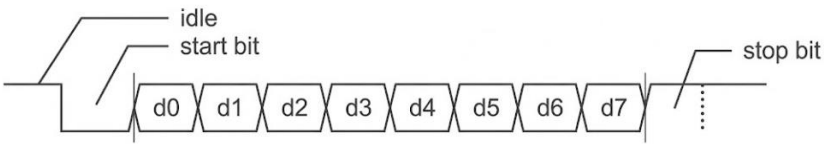


**Figure 2** UART frame (8 data bits, no parity, and 1 stop bit)

No clock information is conveyed through the serial line.

Before the transmission starts, the transmitter and receiver must agree on a set of parameters in advance, which include the ***baud rate*** (i.e., number of bits per second), the number of data bits and stop bits, and use of the parity bit.

The time duration of each bit (***bit interval***) shall be $1/baud\ rate$ (a ***baud period***).

## 3.2 Baude Rate Generator

The baud rate generator generates a sampling signal (`tick`) whose frequency is exactly $16 \times baud\ rate$. This signal is known as the ***oversampling tick*** (***baud tick***).

In every cycle of `tick`, the `tick` signal is asserted for exactly one system clock cycle (the first one).

The baud rate, $b$, is controlled by the system clock frequency, $f_{clk}$, and the run-time configurable divisor, `dvsr`.

$$dvsr = \frac{f_{clk}}{16 \times b} - 1$$

Some commonly used baud rates are 9600, 4800, and 2400, bit per second (bps).

A commonly used clock frequency is 1.8432 MHz, because it can be divided perfectly into standard baud rates with 0% error.

For example, if $f_{clk} = 1.8432\ MHz$ and $b = 9600\ bps$, then

$$dvsr = \frac{1.8432\ MHz}{16 \times 9600\ s^{-1}} - 1 = 11$$

Figure 3 shows a waveform of `tick` in case of $f_{clk} = 1.8432\ MHz$ and `dvsr` = 11 (i.e., $b = 9600\ bps$).

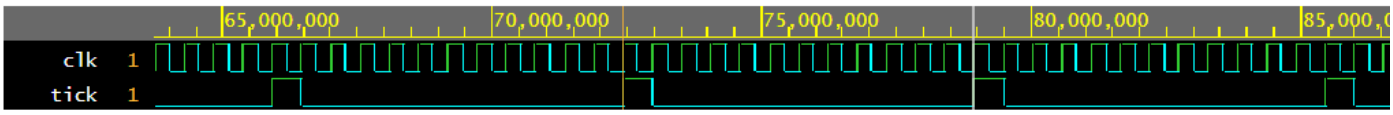


**Figure 3** A waveform of `tick` ($f_{clk} = 1.8432\ MHz$ and $b = 9600\ bps$)

*Note:* From the waveform, we find that a period of `tick` = 6,510,408 $ps$, i.e., its frequency is

$$\frac{1}{6{,}510{,}408\ ps} \approx 153600\ Hz$$

Which equals $16 \times baud\ rate$ (where $baud\ rate = 9600\ bps$).

## 3.3 UART Receiver (Rx)

The receiver performs the serial-to-parallel conversion. It shifts in data bit by bit and then reassembles the data.

The LSB of `dout` is received firstly.

Figure 4 shows a waveform diagram that may be useful to understand the functionality of the Rx. These waveforms are produced by the following code snippet.

```
//initialize
rx <= 1'b1;                     //idle
@(posedge clk);

//drive 1st UART frame
rx <= 1'b0; #BAUD_period;       //start bit

rx <= 1'b1; #BAUD_period;       //data bit 0 (MSB)
rx <= 1'b1; #BAUD_period;       //data bit 1
rx <= 1'b0; #BAUD_period;       //data bit 2
rx <= 1'b1; #BAUD_period;       //data bit 3
rx <= 1'b0; #BAUD_period;       //data bit 4
rx <= 1'b0; #BAUD_period;       //data bit 5
rx <= 1'b1; #BAUD_period;       //data bit 6
rx <= 1'b0; #BAUD_period;       //data bit 7 (LSB)

rx <= 1'b1; #BAUD_period;       //stop bit

//drive 2nd UART frame
rx <= 1'b0; #BAUD_period;       //start bit

rx <= 1'b1; #BAUD_period;       //data bit 0 (MSB)
rx <= 1'b0; #BAUD_period;       //data bit 1
rx <= 1'b0; #BAUD_period;       //data bit 2
rx <= 1'b1; #BAUD_period;       //data bit 3
rx <= 1'b0; #BAUD_period;       //data bit 4
rx <= 1'b1; #BAUD_period;       //data bit 5
rx <= 1'b0; #BAUD_period;       //data bit 6
rx <= 1'b0; #BAUD_period;       //data bit 7 (LSB)

rx <= 1'b1; #BAUD_period;       //stop bit

//wait then finish
#(2*BAUD_period);               //idle
$finish;
```

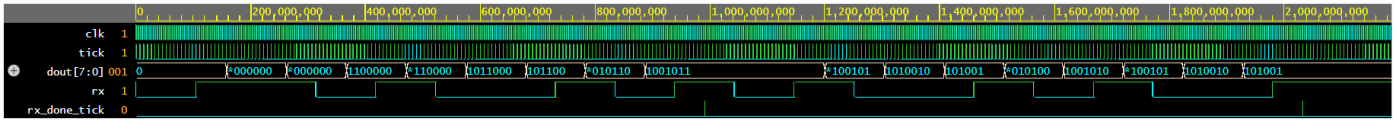


**Figure 4** Receiving two UART frames

## 3.4 UART Transmitter (Tx)

The transmitter performs the parallel-to-serial conversion. It is a special shift register that loads data in parallel and then shifts it out bit by bit.

The LSB of `din` is transmitted firstly.

Figure 5 shows a waveform diagram that may be useful to understand the functionality of the Tx.

Figure 6 and Figure 7 provide expanded views at the first and second `tx_start` pulses, respectively.

These waveforms are produced by the following code snippet.

```
//initialize
tx_start <= 1'b0;              //idle
din      <= '0;

//drive 1st word
@(posedge clk);
tx_start <= 1'b1;
din      <= 8'b1010_1001;      //LSB is transferred firstly

fork
  @(posedge clk) begin
      tx_start <= 1'b0;
      din      <= '0;
  end
join_none

#(10*BAUD_period);             //1 start bit + 8 data bit + 1 stop bit

//drive 2nd word
@(posedge clk);
tx_start <= 1'b1;
din      <= 8'b0110_1110;      //LSB is transferred firstly

fork
  @(posedge clk) begin
      tx_start <= 1'b0;
      din      <= '0;
  end
join_none

#(10*BAUD_period);             //1 start bit + 8 data bit + 1 stop bit

//wait then finish
#(1*BAUD_period);              //idle
$finish;
```

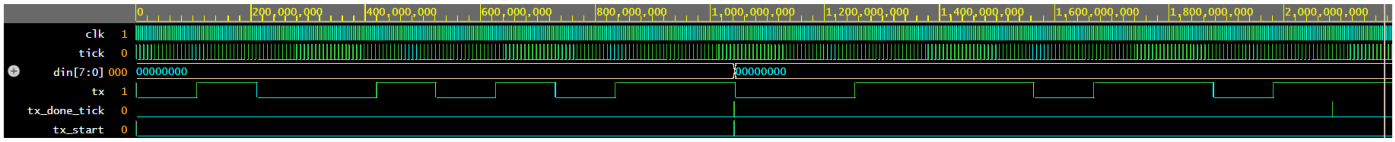


**Figure 5** Transmitting two UART frames

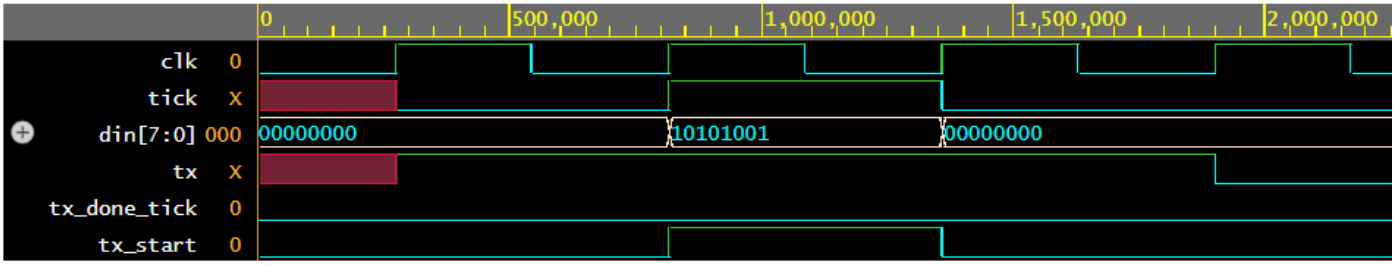


**Figure 6** Expanded view at the first `tx_start` pulse

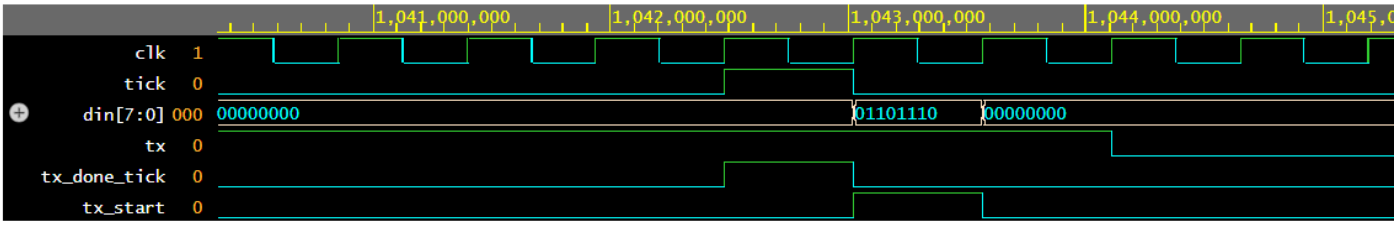


**Figure 7** Expanded view at the second `tx_start` pulse

# 4 References

[**1**] Pong P. Chu, *FPGA Prototyping by SystemVerilog Examples*, Wiley, 2017.

[**2**] Efinix Inc., "UART Core User Guide," [Online]. Available: https://www.efinixinc.com/support/ip/uart.php.